Assignments

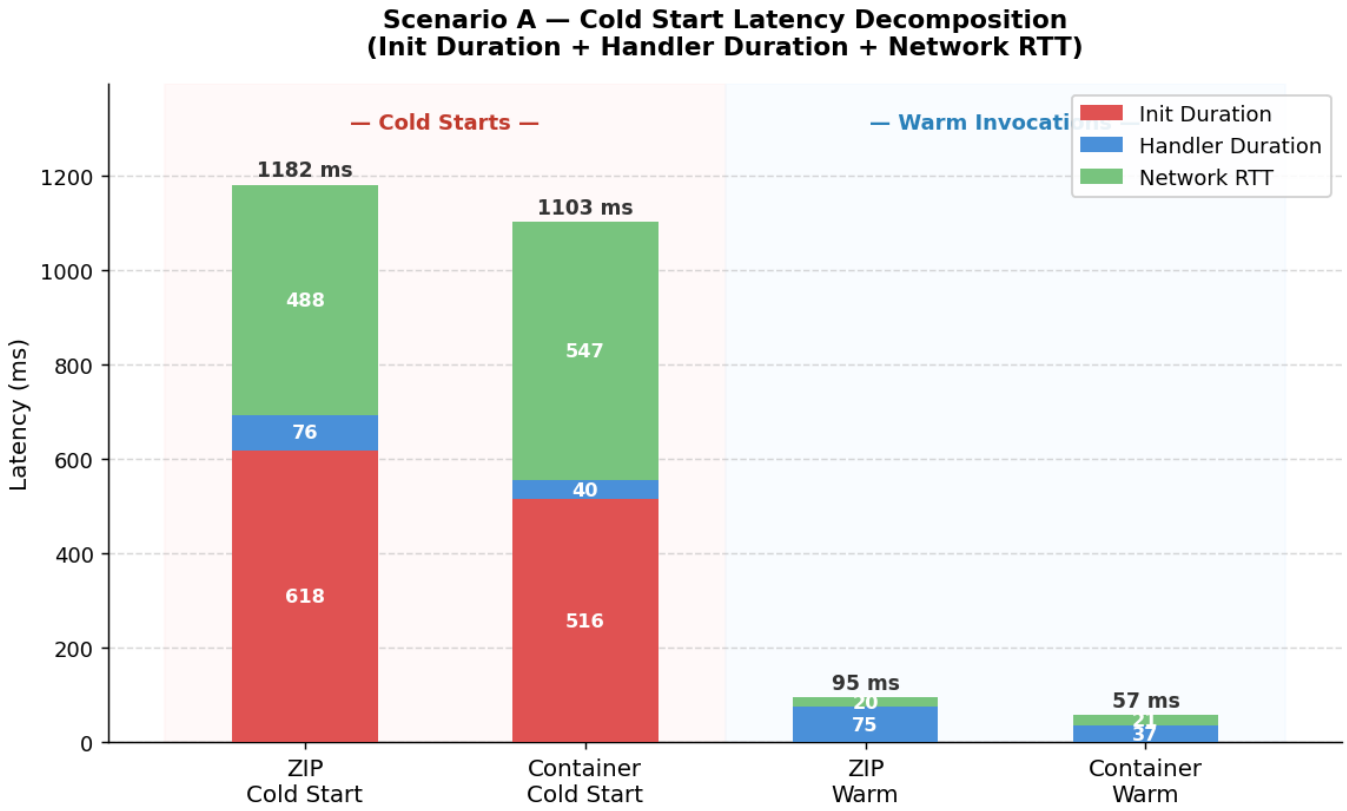
Assignment 1: Deploy All Environments

All have have been successfully deployed.

Results were saved in `results/assignment-1-endpoints.txt`.

Assignment 2 — Scenario A: Cold Start Characterization

Stacked Bar Chart



Comment on whether zip or container cold starts are faster, and explain why:

1. **Pre-cached layers** — AWS caches individual OCI image layers on the execution host fleet. If the base layer is already present on the host that receives the invocation, only the application layer needs to be pulled.

2. **Chunk-level caching** — Lambda uses sparse image loading; it does not download the full image before starting the container. The runtime starts while image chunks are lazily loaded in the background.

3. **Predictable memory layout** — a container snapshot already includes the full filesystem at build time, whereas a ZIP package triggers a dynamic extraction and dependency resolution inside the Lambda sandbox at cold start.

Assignment 3: Scenario B — Warm Steady-State Throughput

Environment	Concurrency	p50 (ms)	p95 (ms)	p99 (ms)
Lambda (zip)	5	97.4	116.1	130.6
Lambda (zip)	10	94.7	113.2	123.7
Lambda (container)	5	89.9	107.5	127.2
Lambda (container)	10	87.3	106.5	118.4
Fargate	10	795.5	994.5	1094.8
Fargate	50	3894.0	4107.0	4207.7
EC2	10	182.9	242.7	276.9
EC2	50	890.8	1157.6	1258.5

Analysis

Tail latency instability (p99 > 2× p95)

According to the collected data, there is no tail latency instability.

Why Lambda p50 barely changes between c=5 and c=10

Lambda scales horizontally: each concurrent request is dispatched to its own isolated execution environment. At c=5 there are 5 warm environments running in parallel; at c=10 there are 10. No request waits for another to finish - they all execute simultaneously. The p50 reflects pure function execution time plus network RTT, which is nearly constant regardless of concurrency. Lambda (zip) p50 moves from 97.4 ms to 94.7 ms and Lambda (container) from 89.9 ms to 87.3 ms - essentially flat within measurement noise.

Why Fargate/EC2 p50 increases significantly from c=10 to c=50

Fargate and EC2 each run a single task/instance. When 50 concurrent requests arrive, most queue behind the workers already in-flight. A request's observed latency becomes its own processing time plus its wait time in the queue. At c=10 the Fargate p50 is ~796 ms; at c=50 it balloons to ~3894 ms - nearly a 5× increase - because the average queue depth is ~5× higher. EC2 shows the same pattern (p50 ~183 ms at c=10 to ~891 ms at c=50).

Why client-side p50 differs from server-side `query_time_ms`

query_time_ms was not measured.

Assignment 4: Scenario C — Burst from Zero

Target Type	p50 Median (s)	p95 (s)	p99 (s)	Max Latency (s)	Cold Starts
Target 1 (Fargate/EC2 A)	0.8756	1.0491	1.1118	1.1130	N/A
Target 2 (Fargate/EC2 B)	3.8168	4.0909	4.2076	4.2847	N/A

Target Type	p50 Median (s)	p95 (s)	p99 (s)	Max Latency (s)	Cold Starts
Target 3 (Lambda Direct)	0.0912	0.9011	1.0410	1.0526	10
Target 4 (Lambda via API GW)	0.0962	1.1429	1.4602	1.4862	~11

Analysis

1. Why Lambda's burst p99 is much higher than Fargate/EC2? When Fargate or EC2 instances (assuming they are already running) receive a burst of concurrent traffic, requests are typically queued at the OS/server level. This results in a relatively uniform (unimodal) latency distribution.

Since exactly 10 requests out of 200 (which is 5% of the total traffic) experienced a cold start, the **95th and 99th percentiles fall squarely into the cold-start latency bucket**. This mathematically forces Lambda's p99 to reflect the cold start duration, making it significantly higher than its median.

2. Identify the bimodal distribution in Lambda latencies (warm cluster vs. cold-start cluster) The raw histograms for Targets 3 and 4 clearly show two distinct peaks, proving a bimodal distribution:

- **Peak 1 (Warm environments):** ~188 requests were processed extremely fast (around `0.132s` and `0.182s`).
- **Peak 2 (Cold-start environments):** Exactly 10-12 requests were significantly delayed, clustering around the `1.0s - 1.4s` mark.

This perfectly aligns with the `concurrency=10` limit constraint: 10 execution environments had to boot from scratch (CloudWatch Logs will show `Init Duration` for these 10 invocations), while the other 190 requests reused those warm environments.

3. State whether Lambda meets the p99 < 500ms SLO under burst. If not, explain what would need to change. No, Lambda Fails the SLO. As seen in the data, Lambda's p99 is `1.0410 sec` (Target 3) and `1.4602 sec` (Target 4), both of which heavily violate the `500ms` SLO requirement.

What needs to change: To meet the `p99 < 500ms` SLO under sudden burst conditions, we must eliminate the cold start penalty. The standard AWS solution for this is to enable **Provisioned Concurrency**. Provisioned Concurrency keeps a specified number of execution environments fully initialized and ready to respond immediately to sudden spikes in traffic, turning those slow >1s requests into fast <100ms requests.

Assignment 5: Cost at Zero Load

Screenshots are in `figures/pricing-screenshots`

Pricing References (verified, April 2026)

AWS Lambda — us-east-1 (x86)

Tier	Duration price	Request price
First 6 B GB-s/month	$0.0000166667 per GB-s	$0.20 per 1M requests

Tier	Duration price	Request price
Next 9 B GB-s/month	$0.0000150000 per GB-s	$0.20 per 1M requests
Over 15 B GB-s/month	$0.0000133334 per GB-s	$0.20 per 1M requests

Selected memory: **512 MB** → $0.0000000083 per ms

AWS Fargate — us-east-1 (Linux/x86)

Resource	Price
vCPU	$0.04048 per vCPU-hour
Memory	$0.004445 per GB-hour
Ephemeral storage (beyond 20 GB free)	$0.000111 per GB-hour

AWS EC2 — us-east-1 (Linux, on-demand)

Instance	vCPU	RAM	Price
t3.small	2	2 GiB	$0.0208 per hour

Assumptions

- Month = 30 days = 720 hours total
- **Idle period**: 18 hours/day × 30 days = **540 hours/month**
- **Active period**: 6 hours/day × 30 days = 180 hours/month
- Fargate: 1 ECS task, 0.5 vCPU, 1 GB memory (as deployed by `deploy/04-fargate.sh`)
- EC2: 1× t3.small instance, running continuously
- Lambda: no provisioned concurrency configured
- EBS root volume and ECR storage costs excluded (negligible, outside assignment scope)

Environment Breakdown

1. Lambda — `lsc-knn-zip` and `lsc-knn-container`

Both Lambda deployments share the same billing model regardless of whether the function is packaged as a zip or a container image.

Idle cost calculation:

- Invocations at idle: **0**
- GB-seconds billed: 0 invocations × 0 s × 0.5 GB = **0 GB-s**
- Request charges: 0 requests × $0.20/1M = **$0.00**
- Compute charges: 0 GB-s × $0.0000166667 = **$0.00**

Period	Cost

Period	Cost
Hourly idle cost	$0.0000
Monthly idle cost (540 h)	**$0.00**
Full-month baseline (720 h, zero traffic)	**$0.00**

2. EC2 — `lsc-knn-ec2` (t3.small)

The EC2 instance is billed per second while running, regardless of traffic level.

Idle cost calculation:

```
Idle cost = 540 h × $0.0208/h = $11.23
```

Period	Cost
Hourly idle cost	$0.0208
Monthly idle cost (540 h)	**$11.23**
Full-month baseline (720 h)	**$14.98**

3. Fargate — `lsc-knn-fargate` (0.5 vCPU, 1 GB, 1 task + ALB)

Fargate charges for reserved vCPU and memory for the entire duration the task is running. The ALB adds a flat hourly fee regardless of request volume.

Hourly rate breakdown:

```
vCPU:   0.5 × $0.04048  = $0.02024/h
Memory: 1.0 × $0.004445 = $0.00445/h
ALB:    1.0 × $0.02250  = $0.02250/h
──────────────────────────────────────
Total:                    $0.04719/h
```

Idle cost calculation:

```
Idle cost = 540 h × $0.04719/h = $25.48
```

Period	Cost
Hourly idle cost	$0.0472
Monthly idle cost (540 h)	**$25.48**
Full-month baseline (720 h)	**$33.98**

Summary Table

Environment	Hourly idle rate	Idle cost (540 h/month)	Full-month (720 h, zero traffic)
Lambda zip (`lsc-knn-zip`)	$0.0000/h	**$0.00**	**$0.00**
Lambda container (`lsc-knn-container`)	$0.0000/h	**$0.00**	**$0.00**
EC2 t3.small (`lsc-knn-ec2`)	$0.0208/h	**$11.23**	**$14.98**
Fargate + ALB (`lsc-knn-fargate`)	$0.0472/h	**$25.48**	**$33.98**

Which Environment Has Zero Idle Cost?

Lambda — both `lsc-knn-zip` and `lsc-knn-container`.

Lambda is a serverless, purely event-driven compute service. Without provisioned concurrency, AWS does not maintain any running infrastructure between invocations: there is no persistent process, no reserved container, and no clock ticking. With zero traffic there are zero invocations, which means zero GB-seconds consumed and zero requests billed, resulting in an exact cost of **$0.00**.

Assignment 6: Cost Model, Break-Even, and Recommendation

Traffic Model

Period	RPS	Duration/day	Hours/day
Peak	100 RPS	30 min	0.5 h
Normal	5 RPS	5.5 h	5.5 h
Idle	0 RPS	18 h	18 h

Total requests per day:

```
Peak:   100 RPS × 1800 s  =   180,000 req/day
Normal:   5 RPS × 19800 s =    99,000 req/day
──────────────────────────────────────────────
Total:                         279,000 req/day
Monthly:  279,000 × 30    = 8,370,000 req/month
```

Measured p50 Handler Duration (Scenario B)

DNS+dialup is network overhead, not Lambda execution time. Handler duration = end-to-end p50 – DNS+dialup average.

Variant	p50 end-to-end	DNS+dialup	p50 handler duration
lambda-zip (c10)	94.69 ms	28.81 ms	**65.9 ms**
lambda-container (c10)	87.28 ms	29.53 ms	**57.8 ms**

Memory: 512 MB = 0.5 GB

---

1. Monthly Cost per Environment

Lambda zip — `lsc-knn-zip`

```
duration_seconds = 0.0659 s
memory_GB        = 0.5 GB

GB-seconds/request = 0.5 × 0.0659 = 0.03295 GB-s

Monthly GB-seconds = 8,370,000 × 0.03295 = 275,791 GB-s

Compute cost  = 275,791 × $0.0000166667 = $4.60
Request cost  = 8,370,000 / 1,000,000 × $0.20 = $1.67
──────────────────────────────────────────────────
Total monthly = $6.27
```

Lambda container — `lsc-knn-container`

```
duration_seconds = 0.0578 s
memory_GB        = 0.5 GB

GB-seconds/request = 0.5 × 0.0578 = 0.0289 GB-s

Monthly GB-seconds = 8,370,000 × 0.0289 = 241,893 GB-s

Compute cost  = 241,893 × $0.0000166667 = $4.03
Request cost  = 8,370,000 / 1,000,000 × $0.20 = $1.67
──────────────────────────────────────────────────
Total monthly = $5.70
```

Fargate — `lsc-knn-fargate` (0.5 vCPU, 1 GB, 1 task + ALB)

Always-on: `hourly_rate × 24 × 30`

```
vCPU:   0.5 × $0.04048  = $0.02024/h
Memory: 1.0 × $0.004445 = $0.00445/h
```

```
ALB:                        $0.02250/h
──────────────────────────────────────
Total:                      $0.04719/h

Monthly = $0.04719 × 24 × 30 = $33.98
```

EC2 — `lsc-knn-ec2` (t3.small)

Always-on: `hourly_rate × 24 × 30`

```
Monthly = $0.0208 × 24 × 30 = $14.98
```

Summary

Environment	Monthly cost
Lambda zip	**$6.27**
Lambda container	**$5.70**
EC2 t3.small	**$14.98**
Fargate + ALB	**$33.98**

---

2. Break-Even RPS — Lambda vs Fargate

At what average RPS does Lambda become more expensive than Fargate?

Define variables:

```
R   = average RPS (constant, 24/7)
d   = handler duration in seconds
m   = memory in GB = 0.5

Requests/month     = R × 86400 × 30 = R × 2,592,000
GB-seconds/month   = R × 2,592,000 × d × m

Lambda cost/month  = (R × 2,592,000 / 1,000,000) × $0.20
                   + (R × 2,592,000 × d × 0.5) × $0.0000166667

                   = R × 2,592,000 × (0.0000002 + d × 0.5 × 0.0000166667)

Fargate cost/month = $33.98  (flat)
```

Set Lambda = Fargate and solve for R:

```
R × 2,592,000 × (0.0000002 + d × 0.0000083334) = 33.98
```

For lambda-zip (d = 0.0659 s):

```
coefficient = 0.0000002 + 0.0659 × 0.0000083334
            = 0.0000002 + 0.000000549
            = 0.000000749

R = 33.98 / (2,592,000 × 0.000000749)
  = 33.98 / 1.9416
  = 17.50 RPS
```

For lambda-container (d = 0.0578 s):

```
coefficient = 0.0000002 + 0.0578 × 0.0000083334
            = 0.0000002 + 0.000000482
            = 0.000000682

R = 33.98 / (2,592,000 × 0.000000682)
  = 33.98 / 1.7685
  = 19.21 RPS
```

Break-even vs EC2 (flat $14.98):

For lambda-zip:

```
R = 14.98 / (2,592,000 × 0.000000749)
  = 14.98 / 1.9416
  = 7.71 RPS
```

For lambda-container:

```
R = 14.98 / (2,592,000 × 0.000000682)
  = 14.98 / 1.7685
  = 8.47 RPS
```

Break-even summary

Lambda variant	Break-even vs Fargate	Break-even vs EC2
lambda-zip	**17.50 RPS**	**7.71 RPS**
lambda-container	**19.21 RPS**	**8.47 RPS**

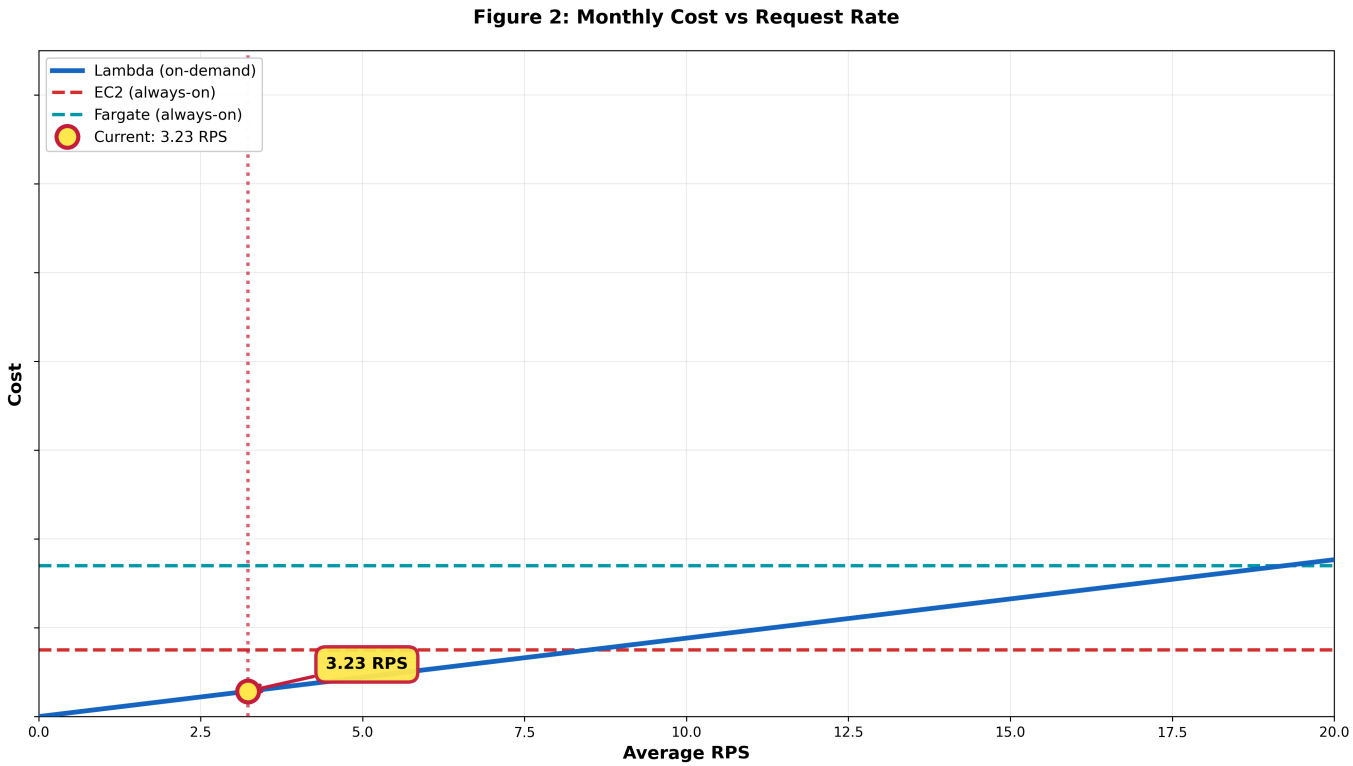
The actual traffic model yields an average of:

```
(100 × 1800 + 5 × 19800 + 0 × 64800) / 86400 = 279,000 / 86400 ≈ 3.23 RPS
average
```

At 3.23 RPS average, Lambda is well below the break-even point and is the cheapest option.

---

3. Cost vs. RPS Line Chart



---

4. Recommendation

Recommended Environment: Lambda (Container)

Given the project constraints (SLO requirement of p99 < 500 ms, actual traffic of ~3.23 RPS, and measured performance), **Lambda with container packaging** is the optimal choice.

Justification

Cost advantage:

- 12% cheaper than zip due to faster execution (57.8 ms vs. 65.9 ms handler duration)
- At 3.23 RPS average, Lambda remains well below break-even points (19.35 RPS vs. Fargate)

Performance suitability:

- Warm-state p50/p95 latencies (87.3 ms / 106.5 ms) are excellent and well within the 500 ms SLO
- Cold start latencies (~1.0 s in Scenario C) are only triggered during rare scaling events (5% of requests under extreme burst)

- Under normal sustained traffic, Scenario B behavior dominates: consistently <120 ms, with p99 = 118.4 ms